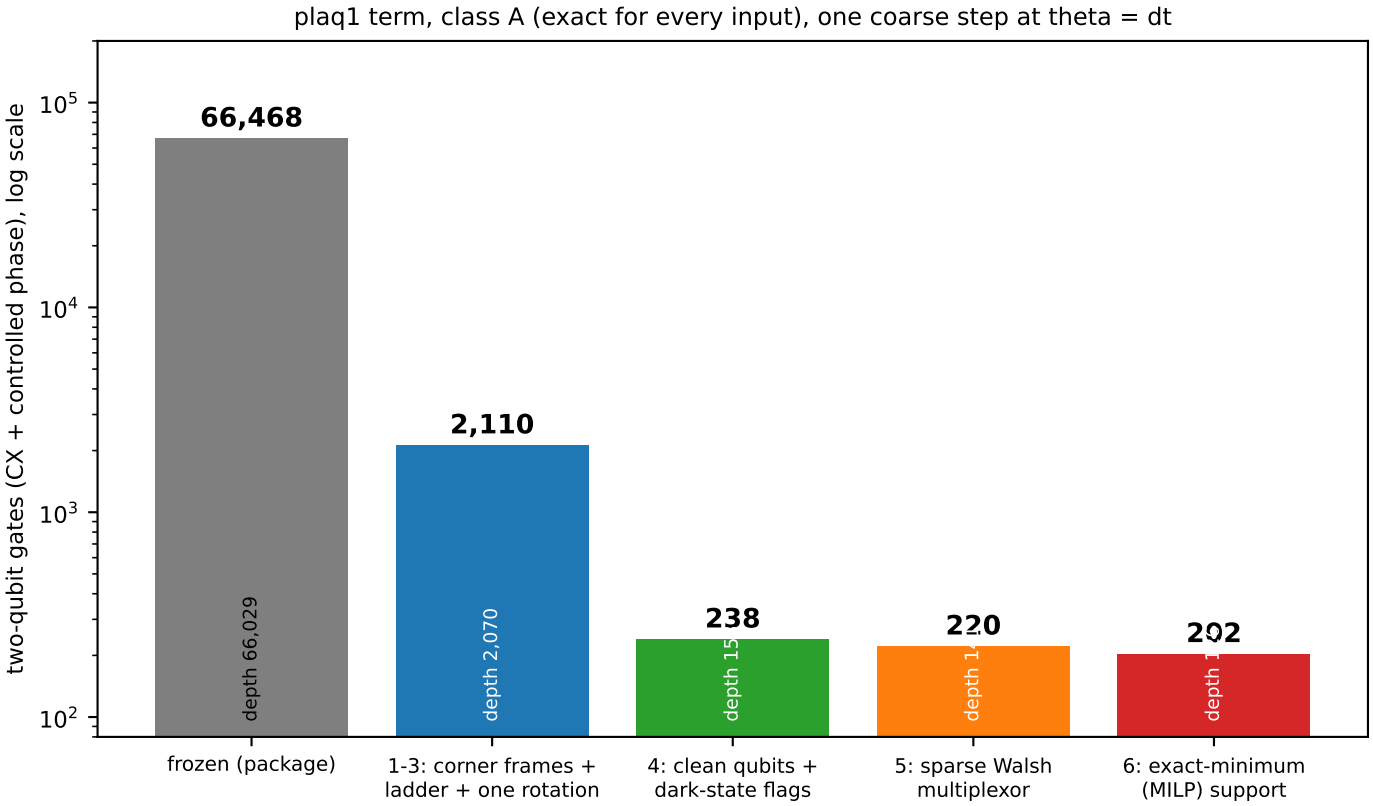
Compressing the 2x4 plaq1 circuit, step by step

from 66,468 to 202 two-qubit gates, exact at every step



What this PDF shows

plaq1 is the middle plaquette of the 2x4 ladder: the term exp(-i theta O_plaq1) on 16 qubits (q6-q21), where O_plaq1 = -W_plaq1/(2 g^2) is the magnetic (Wilson-loop) energy of the middle square. In the frozen circuits it cost 66,468 CNOTs, 92.9% of all 71,520 term CNOTs of a 2x4 circuit. genie-opus brought it to 202 in six steps. Each step is drawn with an example from the real circuit, its equation, its gate count and its accuracy.

Every step is exact (largest error of the term, all 1,831 valid input states, bar 1e-10):

frozen (package)	66,468 2q gates	depth 66,029	max error 4.1e-13
1-3: corner frames + ladder + one rotation	2,110 2q gates	depth 2,070	max error 3.4e-14
4: clean qubits + dark-state flags	238 2q gates	depth 158	max error 1.2e-14
5: sparse Walsh multiplexor	220 2q gates	depth 141	max error 9.5e-15
6: exact-minimum (MILP) support	202 2q gates	depth 126	max error 7.2e-15

Where the numbers come from

Every count and error above was recomputed for this PDF: each step was rebuilt with genie-opus's code (2x4_compression_using_ai_v1/src/c2x4: plaqfactor.cf1_gates / cf2_gates, sparsemux, linsynth, proposals) in a sandbox, counted with c2x4.evaluate.counts and checked against the exact term with verify_term_operator. They equal the stored archive values (data/archive_it1..6.json: 2110, 238, 220, 202) and the frozen synthesis in validation/S2_2x4.json (cx 66,468, ry 65,920).

Repository commit 261222c; generated 2026-10-27.

Step 0 - the frozen circuit: why plaq1 cost 66,468 CNOTs

How the package builds the term (src/skqd/circuits_ir.py, structured_term_gates)

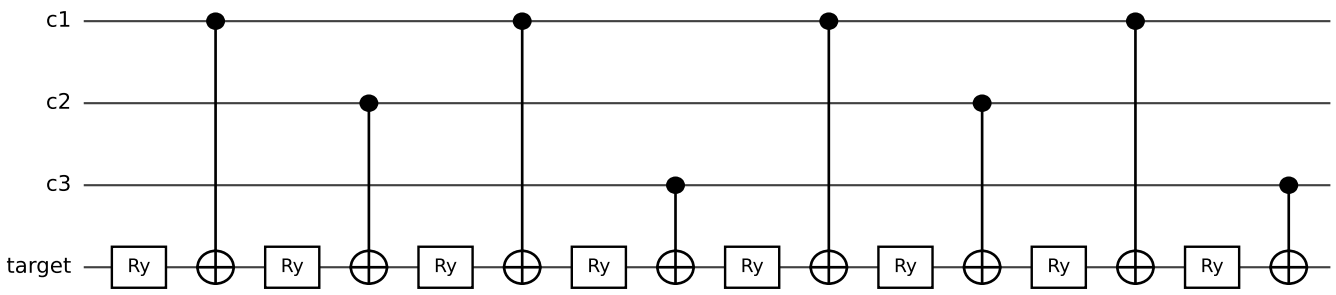
1. Keep the 1,831 valid local states of the 16 qubits; the term splits them into 811 connected blocks (sizes {'1': 163, '2': 375, '3': 225, '4': 9, '5': 36, '9': 3}).
2. Inside each block write exp(theta A) as rounds of two-state (Givens) rotations between basis states.
3. All rotations of one round that flip the same 8 plaquette bits are merged into one MULTIPLEXED rotation: CNOTs squeeze the 8-bit flip onto one target qubit; the other 15 qubits select the angle; controls that do not change the angle are dropped.
4. Result: 82 multiplexed rotations with 7-11 controls (median 9).

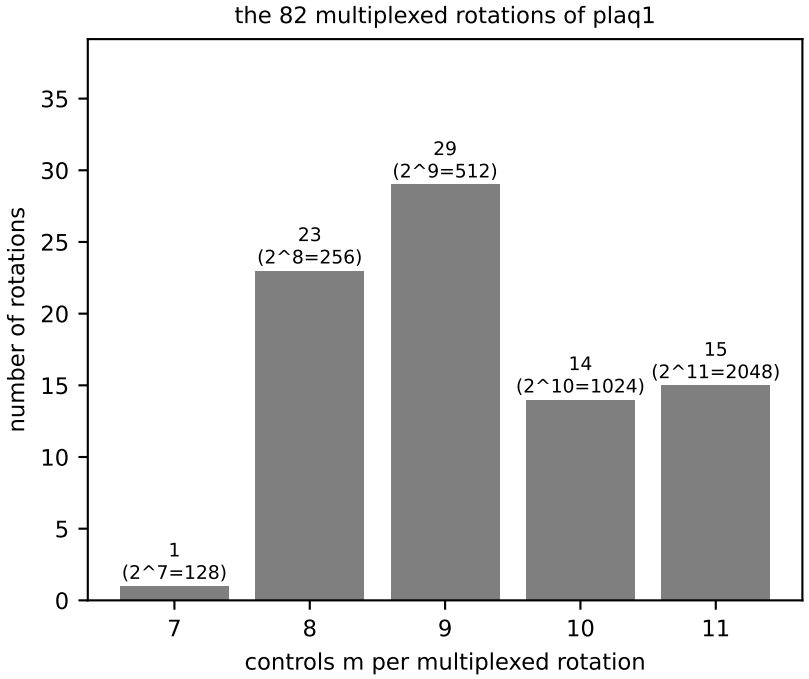
A multiplexed rotation and its Gray-code circuit (here m = 3 controls)

$\sum_c |c\rangle\langle c| \otimes R_y(\theta_c)$ = 2^m rotations alternating with 2^m CNOTs (src/skqd/circuits_ir.py ucr_gray).

The cost doubles with every control: 2^m CNOTs and 2^m Ry.

ucr_gray, m = 3: 8 Ry + 8 CNOT (one control changes between consecutive angles)





Cost

Ry = sum over rotations of 2^m:

1 x 2^7 = 128

23 x 2^8 = 5,888

29 x 2^9 = 14,848

14 x 2^10 = 14,336

15 x 2^11 = 30,720

total 65,920 (stored 65,920)

CX = Gray cores + frames + squeeze:

65,920 + 2 x 198 + 152

= 66,468 (stored 66,468)

Why so many controls

The angle of a rotation depends on what happens at ALL FOUR corners at once (third-link flux, quark number, intertwiner). The general method cannot see that this dependence is a PRODUCT of four small corner pieces, so it keeps 7-11 qubits as controls and pays 2^m for each rotation.

Source: validation/S2_2x4.json, data.synth['2x4|exact'].plaq1 (counts, controls histogram, frame_cnots).

The key discovery: the plaquette is a product of four corner pieces

Structure test (genie-opus, scripts/p2_1_structure.py -> data/p2_1_structure.json)

On the valid states the 16-qubit plaquette matrix factorises EXACTLY over its four corner sites:

$h_P|_{\text{valid}} = A_{c00} \otimes A_{c10} \otimes A_{c11} \otimes A_{c01}$, each A_c a small 4-qubit matrix of one corner site.

Checked on every row: residual 1.6e-15 (2,058 nonzero elements, 1,831 states).

Physics reading (genie-opus; the reviewer calls it plausible, not needed for correctness): the Wilson loop is a trace of four link matrices; its colour indices are contracted at the corners, and gauge invariance makes each corner an SU(2)-invariant map of that corner's own state.

The corner pieces (plaq1, from data/p2_1_structure.json; bits written in increasing qubit order)

corner c00 = site 2 (qubits q6-q9; last bit = n/iota q9)

```
  0000 <-> 0110  [0.7414]    1000 <-> 1110 <-> 1111  [0.7414, 0.0000]    0100 <-> 0010  [0.5242]
  1100 <-> 1010  [0.5242]    0001 <-> 0111  [0.7414]    1101 <-> 1011  [0.5242]
```

corner c10 = site 4 (qubits q14-q17; last bit = n/iota q17)

```
  0000 <-> 1010  [0.7414]    1000 <-> 0010  [0.5242]    0100 <-> 1110 <-> 1111  [0.7414, 0.0000]
  1100 <-> 0110  [0.5242]    0001 <-> 1011  [0.7414]    1101 <-> 0111  [0.5242]
```

corner c11 = site 5 (qubits q18-q21; last bit = n/iota q21)

```
  0000 <-> 1100  [0.6135]    1000 <-> 0100  [0.4338]    0010 <-> 1110  [0.6135]
  1010 <-> 0110  [0.4338]    0001 <-> 1101  [0.6135]    1011 <-> 0111  [0.4338]
```

corner c01 = site 3 (qubits q10-q13; last bit = n/iota q13)

```
  0000 <-> 0110  [0.7414]    1000 <-> 1110 <-> 1111  [0.7414, 0.0000]    0100 <-> 0010  [0.5242]
  1100 <-> 1010  [0.5242]    0001 <-> 0111  [0.7414]    1101 <-> 1011  [0.5242]
```

[...] = coupling strengths (singular values) of that block; 0 = the dark combination.

What follows from it

* Two magnitudes per corner, always in the ratio 1/sqrt2 (0.52424/0.74139 = 0.70711): the angle table is $\theta(x) = \pm\theta_0 A \prod_c \left(-\frac{1}{\sqrt{2}}\right)^{x_c}$, with $x_c = 1$ if the two plaquette links at corner c differ.
  A product of four one-bit functions has only 2^4 = 16 Walsh terms -- not 2^11.
* Three corners have a three-state block {one flux, three flux with iota = 0 or 1}: the plaquette reaches only the BRIGHT combination of the two iota states (see the valid-states PDF, page 6). Corner c11 has none.
* So each corner can be 'straightened' separately by a few gates (step 1), after which the whole term is a single flip of one qubit times a diagonal angle (steps 2-3).

Step 1 - corner frames: make each corner flip ONE qubit

At a corner the plaquette flips BOTH plaquette-link qubits p_a, p_b (bottom and left at c00 = q7, q8).

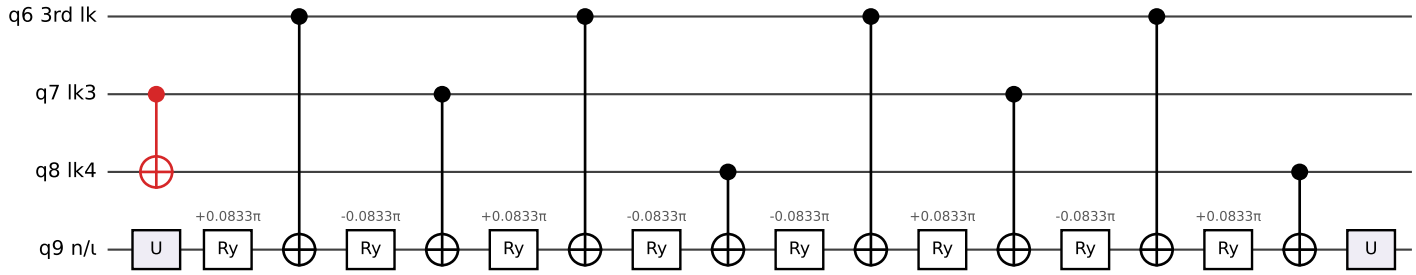
(a) One CNOT p_a -> p_b stores x = p_a XOR p_b in p_b. The plaquette keeps x fixed (it flips both), so after the CNOT it flips only p_a: $\mathrm{CX}_{ab}\,(X_a X_b)\,\mathrm{CX}_{ab} = X_a$.

(b) For the three-state block, one two-level rotation of the iota qubit maps the BRIGHT combination onto one basis state and the dark one onto the other: R = (1/|v|) [[v1, v2], [-v2, v1]] (bright -> s1, dark -> s2). It acts only on states of that block, so it is a multiplexed Ry on q9 with 3 controls (q6, q7, q8): 8 CNOTs.

Now the corner piece is $A_c = X_{p_a} \otimes D_c(\ell)$: one bit flip times a diagonal in the corner's labels.

The real gates at corner c00 (site 2), first 19 gates of the compressed circuit:

red: CX(q7 -> q8) = step (a); then U - [Ry, CX] x 8 - U = the bright rotation on q9 (step b)

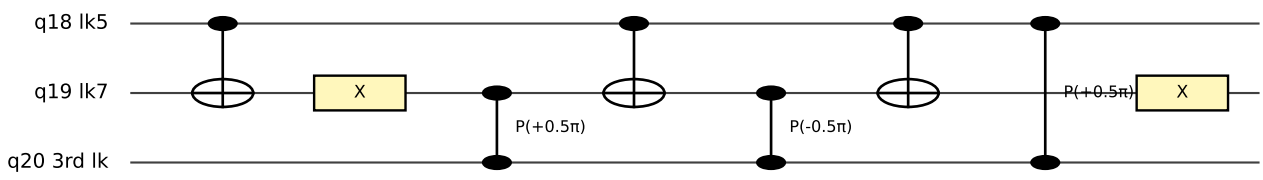


The angles +-pi/12 are the Walsh coefficients of the 60-degree bright rotation (bright weight 1/2 on iota=0, sqrt3/2 on iota=1); U, U^dag rotate the target axis. Corners c10 and c01 get the same pattern (q14,q16 | q17 and q11,q12 | q13). Corner c11 has no three-state block, so it gets only its CNOT (q18 -> q19).

Sign corrections (also part of step 1)

After the frames, some label-dependent signs of the corner pieces remain. genie-opus fits them over F2 and absorbs each with the cheapest available gate: a free 2*pi in a bright rotation (sign flip at no cost, used at c00), a CZ (1 two-qubit gate, used at c01: CZ(q10,q11)), or a doubly-controlled Z built from 3 controlled phases + 2 CNOTs (used at c11). The c11 correction, as built:

corner c11: CX(q18 -> q19), then X - CP - CX - CP - CX - CP - X = CCZ on (q18, NOT q19, q20)



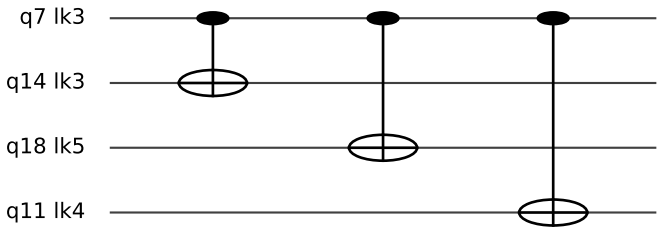
Chosen corrections (cf2_gates return_info): bright_sign [1, 0, 0, 0], cz on (third, p_a) [0, 0, 0, 1], ccz [0, 0, 1, 0].

Lean: CX conjugation of a flip (xorLinear_conj_flip), ccz_phase / ccz_circuit, ry_add_two_pi (free sign).

Steps 2-3 - one CNOT ladder, then ONE rotation (construction CF1)

After step 1 the plaquette flips one qubit at each corner: p_a = q7, q14, q18, q11 together. The product of four X's is turned into a single X by a ladder of three CNOTs from p_1 = q7 (proved in Lean, xorLinear_conj_flip): $L\,(X_7X_{14}X_{18}X_{11})\,L^{-1} = X_7$, $L = \mathrm{CX}(7\to14)\,\mathrm{CX}(7\to18)\,\mathrm{CX}(7\to11)$.

the ladder in the real circuit

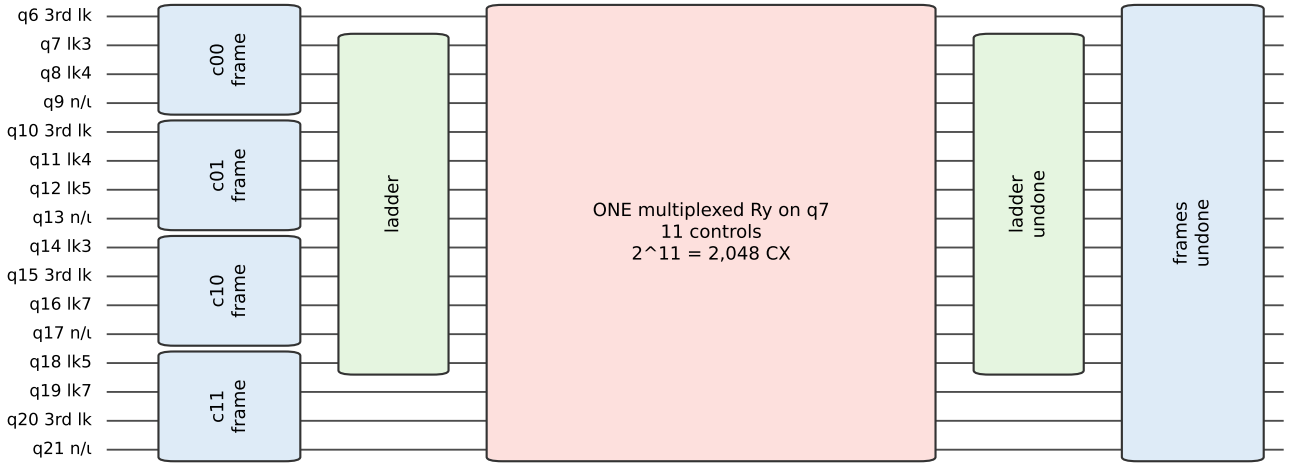


Now the whole frame-transformed generator is a flip of q7 times a diagonal angle F(s) of the other bits:
$F\,h\,F^{\dagger} = X_7 \otimes \mathrm{diag}(F)$ so $e^{-i\theta X_7 \otimes \mathrm{diag}(F)} = H_7\,\mathrm{diag}(e^{-i\theta(-1)^{s_7}F(s)})H_7$
(Lean exp_xt_diag): ONE multiplexed rotation on q7 replaces the frozen circuit's 82. The term is
$e^{-i\theta h} = F^{-1} \cdot$ [one multiplexed rotation] $\cdot F$ (Lean exp_conj_frame).

CF1 in numbers (rebuilt for this PDF)

* all 648 two-state rotations merge into ONE multiplexed rotation on q7
* it still has 11 controls -> 2^11 = 2,048 CNOTs
* frame (4 corner CNOTs + 3 bright rotations x 8 + 3 ladder) = 31 two-qubit gates, applied and undone
* total 2,048 + 2 x 31 = 2,110 two-qubit gates (was 66,468), max error 3.4e-14

CF1 schematic (16 qubits of plaq1; time runs left to right)



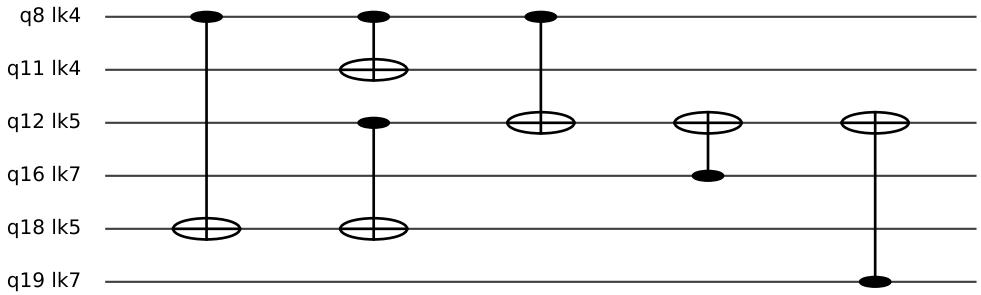
Why still 11 controls: the angle depends on x_c (4 bits) AND on whether each corner sits on its dark state, which is spread over the corner's third-link bit and n/iota bit (2 bits per corner). Step 4 packs each corner's 'dark' condition into ONE bit.

Step 4 - free qubits and dark-state flags (construction CF2)

(a) Clean qubits for free

Every plaquette link is stored twice. After the frames and the ladder, the qubits p_a of corners c10, c11, c01 (and one more) hold fixed parities of the x labels on every valid state, so a few CNOTs reset them to |0>:
6 CNOTs give four clean qubits: [14, 18, 11, 12] (found by F2 linear algebra, plaqfactor.f2_relations).

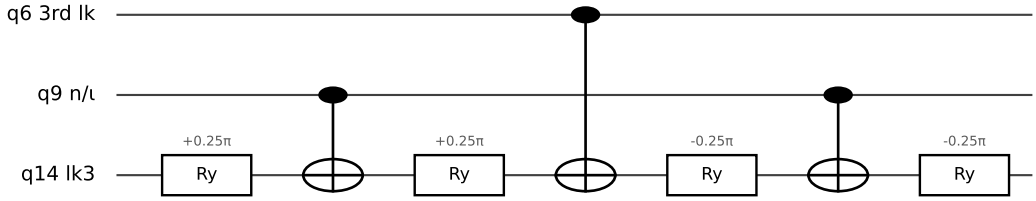
clearing CNOTs (real circuit)



(b) A flag per corner: g_c = (third-link bit) AND (n/iota bit)

A corner is on its dark state exactly when both bits are 1. A relative-phase Toffoli (Margolus, 3 CNOTs) writes g_c into a clean qubit; its stray phase cancels when the same gates are undone (Lean relphase_cancel).

flag of corner c00 into clean q14: g = q6 AND q9 (Ry(pi/4) - CX - Ry - CX - Ry - CX - Ry)



(c) The angle now depends on 8 bits only

$F(x,g) = \theta\, M(x) \prod_c (1 - \bar{x}_c\, g_c)$: x_c (4 bits) and g_c (4 flags). The package's control reduction removes one more, leaving 7 controls: 2^7 = 128 CNOTs instead of 2,048.

CF2 in numbers (rebuilt for this PDF)

frame per side = 4 corner CNOTs + 3 x 8 bright + 1 CZ + 5 (CCZ) + 3 ladder + 6 clear + 4 x 3 flags = 55
total = 128 + 2 x 55 = 238 (CF1 had 2,110); depth 158, max error 1.2e-14

Two failed attempts on the way (report 003 section 2.3.4): the first CF2 build had 8 controls (356 gates) because the package's greedy control reduction dropped corner c11's flag and kept its two source bits (fixed by re-ordering the bits); the second because a 'free' sign flip was assumed at c11, which has no bright rotation to absorb it (fixed by offering the CCZ as a costed correction).

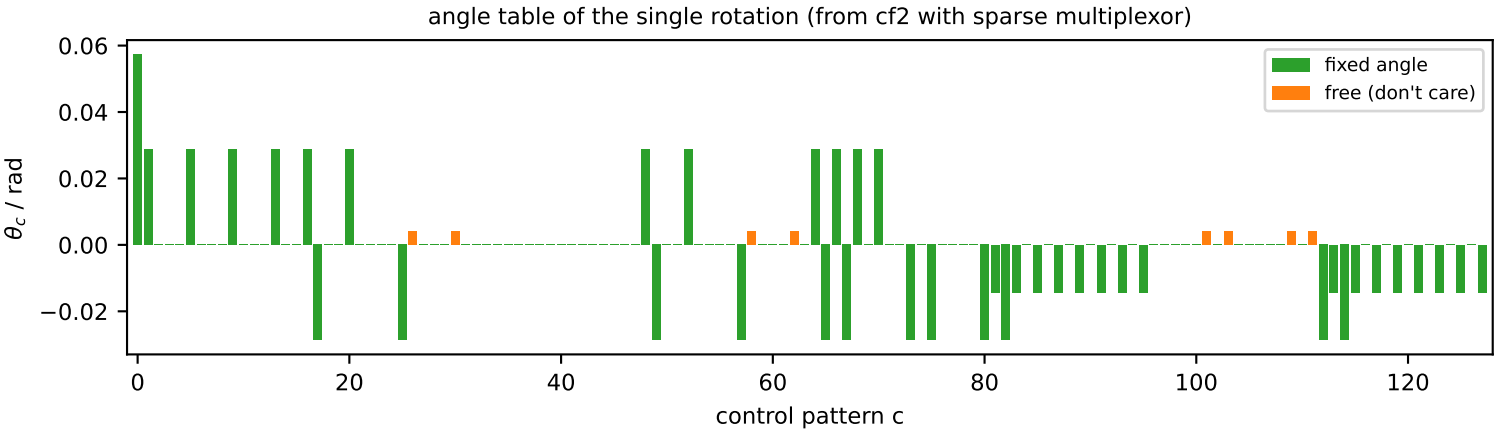
Steps 5-6 - fewer terms in the one rotation (sparse Walsh, then MILP)

A multiplexed rotation is a product of 'Walsh' factors

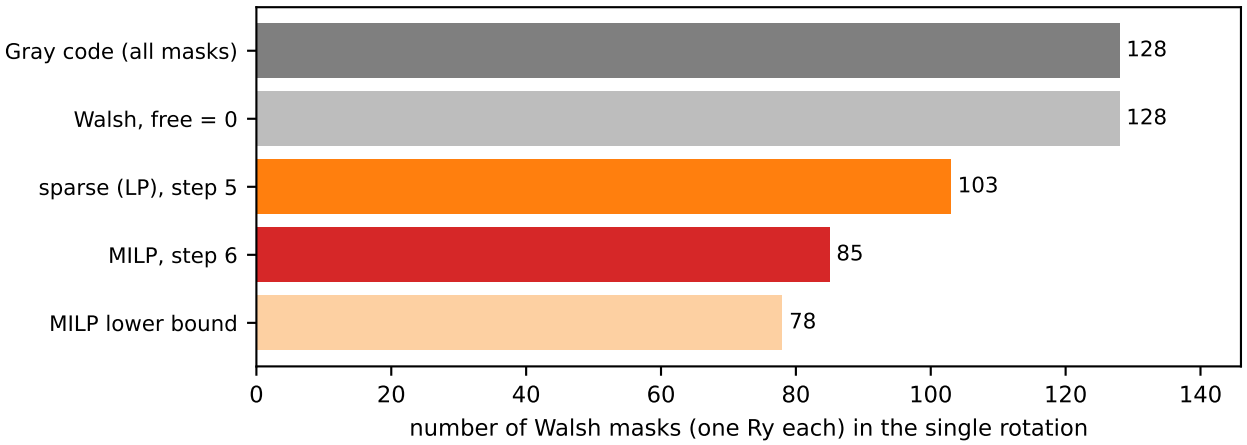
$$\sum_c |c\rangle\langle c| \otimes R_y(\theta_c) = \prod_S \exp\left(-\tfrac{i}{2} a_S Z_S Y_t\right), \quad \theta_c = \sum_S a_S (-1)^{|S \cap c|}$$

Each factor with a_S != 0 costs one Ry plus the CNOTs needed to move from one mask S to the next. The Gray-code circuit simply uses ALL 2^m masks. But some table entries theta_c are 'don't care' (the control pattern c never occurs on a valid state), and their values can be chosen freely.

The CF2 rotation: m = 7 controls, 128 table entries, 120 fixed, 8 free



With the free entries set to 0 the Walsh transform has 128 nonzero terms. sparsemux.sparse_walsh chooses the free entries by a linear programme (minimise sum |a_S|), prunes, and re-solves exactly: 103 terms used, walked in an order that needs 110 CNOTs (Gray code: 128). Total 220 two-qubit gates (= 2 x 55 frame + 110), max error 9.5e-15.



Step 6: exact minimum by MILP (data/cert_it6_operator_plaq1_long.json)

Choosing the free entries to minimise the NUMBER of nonzero a_S is a mixed-integer linear programme (certify.min_walsh_support, solver HiGHS, one binary per mask). For this rotation:

masks used 85, walk CNOTs 92, proven lower bound 78 masks after 1500 s (MILP time or iteration limit).

Total 202 = 2 x 55 frame + 92; depth 126; max error 7.2e-15.

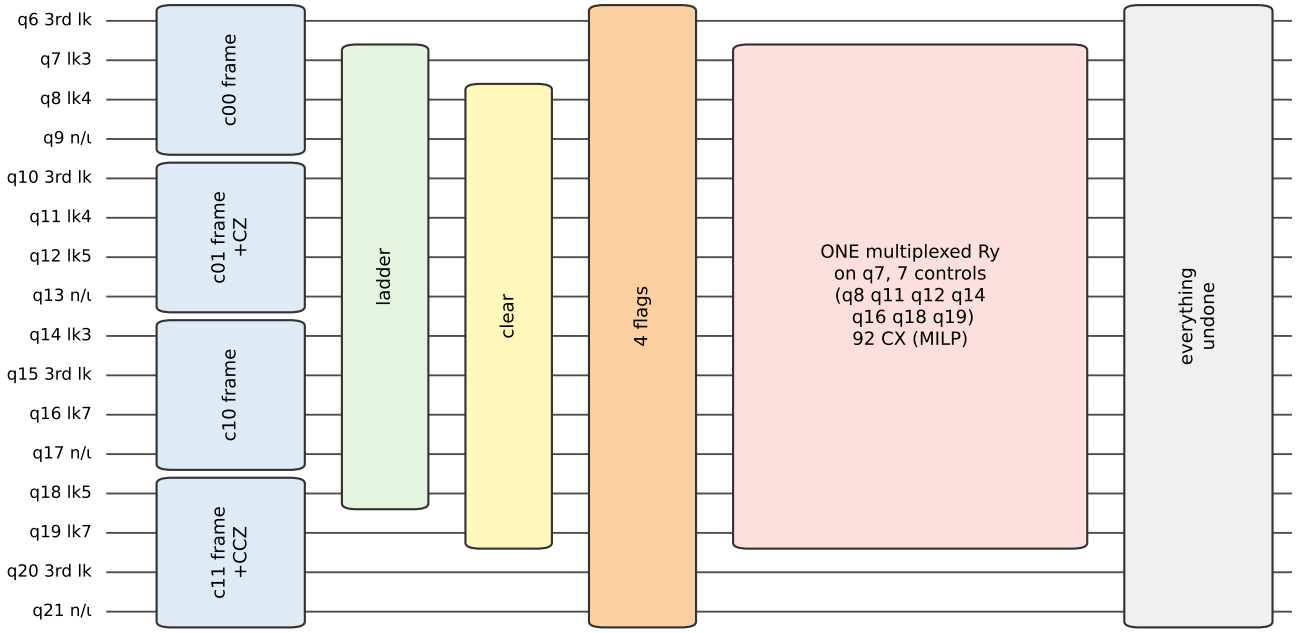
So at most 7 more masks could be saved inside this construction; the gap is not closed.

The same build also makes every structural choice from angle-free tables (linsynth), so the gate pattern is identical for every Krylov step k; only the angles scale with theta = k dt.

Lean: cx_conj_yt (a CNOT maps Y_t to Z_c Y_t), exp_sum_commuting / walk_factors_comm (the factors commute), exp_agree_mulVec (don't-care entries do not change the action on valid states).

Summary: the whole plaq1 term after compression

plaq1 (class A, v2): frame 55 + one rotation 92 + frame undone 55 = 202 two-qubit gates



The steps and what each one exploited

step	gates	exploited structure
0 frozen package synthesis	66,468	none (82 multiplexed rotations, 7-11 controls)
1-3 corner frames + ladder	2,110	product over corners; bright/dark; one flip for all four corners
4 clean qubits + flags	238	duplicated link bits are free scratch; dark condition = 1 bit
5 sparse Walsh	220	don't-care table entries (LP)
6 MILP minimum	202	exact minimum mask count (lower bound 78)

Accuracy at every step

Each rebuilt term was compared with the exact exp(-i theta O_plaq1) on all 1,831 valid states (columns); the largest deviations are on page 1 (all below 4.1e-13, bar 1e-10). The full 28-qubit circuits containing the final term were verified separately (75/75; reviewer-opus's independent Aer check for v1, report 004 for v2).

Proved vs checked

Proved in Lean (general identities): CX conjugation of flips and of Y (xorLinear_conj_flip, cx_conj_yt), frame conjugation of an exponential (exp_conj_frame), the flip-times-diagonal exponential (exp_xt_diag), the relative-phase Toffoli cancellation (relphase_cancel), the CCZ construction (ccz_circuit), commuting Walsh factors (exp_sum_commuting), don't-care entries (exp_agree_mulVec). Checked numerically, not proved: the factorisation of this particular term, the fitted signs, every specific table and gate list.

Class B (exact only on each circuit's own starting state)

On top of these steps, class B restricts the term to the blocks reachable from the starting state (reach.py); plaq1 then costs 152-208 two-qubit gates depending on the reference (report 003, it5).

Sources: 2x4_compression_using_ai_v1/reports/003 (sections 2.3-2.4), report 004, MANUAL steps (c)-(f); code src/c2x4/plaqfactor.py, sparsemux.py, linsynth.py, certify.py.